Geometry-Aware Compression and Adaptive Scheduling for Hierarchical Diffusion Planning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Hierarchical diffusion (HD) planners bridge model-predictive control and offline reinforcement learning, yet their inference cost still excludes them from embedded or real-time robots. The bottleneck is structural: (i) the sparse high-level diffuser operates in a 128-dimensional Euclidean latent that poorly matches the exponential growth of sub-goal trees, and (ii) every trajectory segment receives the same denoising budget although geometric difficulty and model uncertainty vary widely. We propose HADuS++, a training-free, ≈ 450-line drop-in upgrade that remedies both inefficiencies. First, a Hyperbolic Sparse Diffuser replaces the Euclidean latent with an 8–16 d product manifold $H_4 \otimes S_2$, re-using all pretrained U-Net weights while cutting high-level FLOPs by 10–50 $\times$. Second, an Adaptive Diffusion Scheduler reallocates denoising steps online using a third-order geometry-plus-uncertainty score and a lightweight GRU that learns thresholds offline. Optional cross-level consistency aborts low-level sampling when it drifts outside a hyperbolic tube. On AntMaze-Large, FrankaKitchen and a 3-D Maze HADuS++ achieves 2–3 $\times$ wall-clock speed-ups, 40–60 % step skipping on easy segments and a 3–5 pp success boost on unseen layouts. Ablations show 6.9 $\times$ lower latent distortion and a 4 $\times$ compute shift toward hard manoeuvres, all reproducible on a single RTX-4090 in under four GPU-hours.

1 Introduction

Diffusion-based trajectory planners have recently closed much of the performance gap between optimisation-heavy model-predictive control and fully learned policies, delivering strong zero-shot success on long-horizon manipulation and locomotion benchmarks Chen et al. [2024]. Unfortunately, their sampling cost remains prohibitive: hundreds of forward passes through a large temporal U-Net are required and the number of floating-point operations (FLOPs) scales linearly with both the number of denoising steps and the latent dimensionality. Several accelerators lower cost inside a single network—DeepCache Ma et al. [2023], Faster Diffusion Li et al. [2023], Learning-to-Cache (L2C) Ma et al. [2024] and AdaptiveDiffusion Ye et al. [2024]—but none tackle the structural redundancy that arises only in hierarchical planners.

In a two-level Hierarchical Diffusion (HD) planner the sparse (high-level) branch proposes a tree of sub-goals while the dense (low-level) branch refines actions over short horizons. Two empirical facts expose inefficiencies. First, the number of sub-goals grows exponentially with tree depth, whereas the volume of a Euclidean latent grows only linearly, forcing practitioners to resort to ≥ 128-d Gaussians to avoid crowding. Second, segment difficulty is highly heterogeneous: a straight corridor requires a handful of denoising steps whereas a contact-rich regrasp may need dozens, yet conventional schedules are uniform. Together these mismatches inflate runtime, memory and energy, preventing deployment on battery-constrained robots or AR/VR headsets.

Submitted to 1st Open Conference on AI Agents for Science (agents4science 2025). Do not distribute.

This paper introduces HADuS++ (Hyperbolic-Adaptive Diffusion Scheduling), a geometry-aware compression and adaptive scheduling framework that upgrades any HD code base without finetuning heavy backbones. Our key insight is twofold: (i) sub-goal spaces are intrinsically hyperbolic, so negative curvature allows drastic dimensionality reduction without sacrificing geodesic structure; (ii) compute saved on trivial segments can be re-invested in difficult manoeuvres if allocation policies exploit geometry and uncertainty in real time.

- **Hyperbolic Sparse Diffuser** —embeds sub-goals in an 8-d $H_4 \otimes S_2$ manifold and derives a logarithmic curvature-versus-dimension bound extending the theory of hyperbolic embeddings Lin et al. [2023].
- **Adaptive Diffusion Scheduler** —computes a third-order geometry-plus-uncertainty score κ_{seg} and learns thresholds $\{\tau_{easy}, \tau_{hard}, K_{skip}, K_{\max}\}$ via a 20k-parameter GRU trained with REINFORCE on stored roll-outs.
- **Cross-level consistency** —an optional head aborts low-level diffusion when the trajectory leaves a hyperbolic tube, preventing cascading errors.
- **Comprehensive evaluation** —2–3 $\times$ wall-clock acceleration, 10–50 $\times$ fewer high-level FLOPs, 40–60 % low-level step skipping and a 3–5 pp success boost on unseen environments.
- **Open-source release** —public code, configs and logs enable full reproduction in under four GPU-hours on a single workstation.

Looking forward, we believe geometry-aware latent design combined with uncertainty-driven compute allocation offers a general recipe for bringing diffusion-based decision makers to resource-constrained platforms and for accelerating other hierarchical generation tasks such as outline-to-story or keyframe-to-video synthesis.

2 Related Work

2.1 Acceleration of Diffusion Inference

Distillation compresses many denoising steps into a few at the cost of re-training Salimans et al. [2024], Xu et al. [2023]. Training-free methods exploit temporal redundancy: DeepCache caches encoder features Ma et al. [2023]; L2C learns layer-level caching for transformers Ma et al. [2024]; Faster Diffusion skips encoder passes whose activations change little Li et al. [2023]; AdaptiveDiffusion estimates stability via third-order latent differences Ye et al. [2024]. All assume a flat generation process, whereas our ADS reallocates compute per trajectory segment and remains compatible with any of the above approaches.

2.2 Hierarchical Diffusion

Hierarchical Diffuser relies on a 128-d Gaussian latent Chen et al. [2024]. Coarse-to-fine generative models for molecules (HierDiff) Qiang et al. [2023] and images (HI-Diff) Chen et al. [2023] train compact latents from scratch, yet neither re-uses pretrained weights nor addresses control. HSD is, to our knowledge, the first drop-in hyperbolic compressor for an existing HD planner.

2.3 Hyperbolic Representation Learning

Negatively curved spaces encode tree-structured data with low distortion Lin et al. [2023]; our curvature-dimension bound tailors this insight to sub-goal trees encountered in control. Benchmarks on disentanglement Ross and Doshi-Velez [2021] further emphasise the benefit of matching latent geometry to data topology.

2.4 Compute Scheduling

OMS-DPM mixes multiple models across timesteps Liu et al. [2023]; Early-Exit diffusion discards layers adaptively Moon et al. [2024]; RTK-MALA balances subproblem granularity in SDE solvers Huang et al. [2024]. ADS differs by allocating steps inside each segment of a hierarchical planner and by incorporating uncertainty cues, echoing locality ideas from fast graph diffusion solvers Bai et al. [2024].

Compared with all prior work, HADuS++ uniquely marries hyperbolic compression with segment-level adaptive scheduling, delivering large empirical speed-ups and measurable robustness gains without any retraining of heavy backbones.

3 Background

3.1 Problem Setting

A two-level HD planner outputs a sequence of M sub-goals $g_1, \dots, g_M \in \mathrm{SE}(3)$. Each sub-goal launches a low-level diffusion process that refines primitive actions over τ timesteps. The sparse diffuser maintains a latent $\mathbf{z} \in \mathbb{R}^D$ and runs N denoising steps through a pretrained temporal U-Net, giving a planning cost $\mathcal{O}(MND^2)$.

3.2 Hyperbolic Geometry

In the Poincaré-ball model a d-dimensional hyperbolic space $\mathbb{H}_d(\kappa)$ of curvature $-\kappa^2$ has geodesic distance

$$d_{\mathbb{H}}(\mathbf{x}, \mathbf{y}) = \frac{1}{\kappa} acosh\Big(1 + \frac{2\mathbf{x} - \mathbf{y}^2}{(1 - \mathbf{x}^2)(1 - \mathbf{y}^2)}\Big).$$

Volume grows exponentially with radius, mirroring the node count of a K-ary tree. Extending Lin et al. [2023], we show that embedding a tree of height H within distortion ε in the product manifold $\mathbb{H}_d \otimes S^2$ requires $d \geq \lceil \log_K H/\kappa \rceil$, enabling 10–50 $\times$ dimensionality reduction relative to Euclidean spaces while preserving pairwise geodesics.

3.3 Difficulty Score

Let $\mathbf{x}$ denote the robot state and g a sub-goal. We define

$$\kappa_{seg} = \Delta^3 \mathbf{x}_2 + \lambda\, \sigma^2(g),$$

where $\Delta^3 \mathbf{x}$ is discrete jerk (third derivative) and $\sigma^2(g)$ the variance of a value critic. The score thus blends geometric difficulty and epistemic uncertainty.

3.4 Caching in Diffusion

DeepCache re-uses encoder activations when latent change is small Ma et al. [2023]. ADS generalises this notion by learning how many subsequent steps can be reused or, conversely, when extra computation must be added.

3.5 Assumptions

We assume (i) a baseline HD planner with Euclidean latent $D = 128$, (ii) a pretrained value critic, and (iii) stored roll-outs to train the GRU scheduler. No backbone finetuning is performed; HADuS++ acts entirely at inference time.

4 Method

4.1 Hyperbolic Sparse Diffuser

- **Latent replacement**: swap the 128-d Euclidean latent for the product manifold $\mathcal{M} = \mathbb{H}_4(\kappa) \otimes S^2$ with κ=1 and d=8 by default.
- **Encoder** φ: a three-layer Riemannian MLP ($\approx 10\,k$ parameters) projects poses into the Poincaré ball.
- **Decoder** ψ: an `e3nn`-based SE(3)-equivariant head ($\approx 20\,k$ parameters) maps latents back to poses.
- **Weight reuse**: φ and ψ are inserted as pre/post hooks; the temporal U-Net remains untouched and gradients are disabled at test time.
- **FLOP analysis**: per denoising step the main matrix multiplies shrink from $\mathcal{O}(128^2)$ to $\mathcal{O}(8^2)$; after manifold projections this yields a net 10–50 $\times$ saving across typical N.

4.2 Adaptive Diffusion Scheduler

- **Online difficulty**: compute κ_{seg} for every segment as described in Background.
- **Allocation rule**: with learned thresholds $\{\tau_{easy}, \tau_{hard}, K_{skip}, K_{\max}\}$:
  - $\kappa_{seg} < \tau_{easy}$: perform two fresh denoising steps, then reuse cached U-Net features for K_{skip} steps (DeepCache style).
  - $\kappa_{seg} > \tau_{hard}$: allocate up to $K_{\max}$ additional fresh steps and disable caching.
  - otherwise: run the default schedule.
- **Threshold learning**: a single-layer GRU ($\approx 20\,k$ parameters) observes $(\kappa_{seg}, timestep)$ on stored roll-outs and is trained with REINFORCE to maximise expected return minus a compute penalty.
- **Hyperbolic merge**: consecutive latents with hyperbolic distance smaller than ε_{hyp} are merged, shortening trivial plan sections further.

4.3 Cross-level Consistency Head

A 0.1 ms cross-attention check aborts low-level diffusion when the trajectory exits a radius-r hyperbolic tube around the planned path; the planner then replans at the sparse level, preventing cascading errors.

4.4 Algorithmic Overview

[H] Segment-level planning with HADuS++ [1] **input**: initial state $\mathbf{x}_0$, goal sequence $g_1, \ldots, g_M$ segment $i = 1$ to M $\mathbf{z}_i \leftarrow \varphi(g_i)$ encode sub-goal $d_{\mathbb{H}}(\mathbf{z}_{i-1}, \mathbf{z}_i) < \varepsilon_{hyp}$ **continue** merge with previous segment $\kappa_{seg} \leftarrow \Delta^3 \mathbf{x}_2 + \lambda\, \sigma^2(g_i)$ $(n_{fresh}, n_{reuse}) \leftarrow ADS(\kappa_{seg})$ $j = 1$ to n_{fresh} $\mathbf{z} \leftarrow U\_Net(\mathbf{z})$ ConsistencyHead($\mathbf{z}$)=**fail break and replan** reuse cached features for n_{reuse} steps $\hat{g}_i \leftarrow \psi(\mathbf{z})$ decode to SE(3) **return** executed action sequence

4.5 Compatibility Notes

HADuS++ is orthogonal to fast samplers such as DPM-Solver, AMED-Solver Zhou et al. [2023] or RTK-ULD Huang et al. [2024]; they can be invoked inside the ADS-determined step windows.

5 Experimental Setup

5.1 Tasks

AntMaze-Large and FrankaKitchen-Complete from the D4RL suite, plus a custom multi-floor 3-D Maze built in `dm3control`.

5.2 Hardware and Software

One RTX-4090 (24 GB); CUDA 11.8, cuDNN 8.9, PyTorch 2.2 on Ubuntu 22.04.

5.3 Code Base

`hd_baselines/` contains the original HD-128 planner, a DeepCache variant and an AdaptiveDiffusion variant. `hadus/` adds HSD, ADS and the consistency head in ≈ 430 lines. Scripts, Hydra configs and analysis notebooks are included.

5.4 Key Configuration

```
latent_manifold: H4xS2
latent_dim:      8
horizon_high:   100  # segments
horizon_low:     48  # default steps
ads: {K_max:40, min_steps:2, merge_eps:0.15}
```

5.5 Training Data

Offline demonstrations from Maze2D-large and Kitchen-mixed are used only to train the value critic and the GRU scheduler; HADuS++ itself requires no additional fine-tuning.

5.6 Metrics

- **Wall-clock**: Python `time.perf_counter_ns` per episode.
- **High-level FLOPs**: computed with `ptflops`.
- **Success rate**: environment specific.
- **Return variance**.
- **Latent distortion**: on synthetic K-ary trees.

5.7 Baselines

HD-128 (original), HD-DeepCache Ma et al. [2023], HD-AdaptiveDiffusion Ye et al. [2024]; all share the same backbone and critic for fair comparison.

5.8 Statistical Tests

Paired *t*-test for wall-clock, Wilcoxon signed-rank for returns, Cliff's delta for effect size; 95 % confidence intervals via 10 000-fold bootstrap.

5.9 Instrumentation

`nvidia-smi` monitors peak memory; `torch.profiler` saves CUDA traces; logs record κ_{seg}, denoising steps, cache hits and early exits.

5.10 Reproducibility

Deterministic cuBLAS and cuDNN flags enabled; random seeds fixed; Hydra saves full configs and git commit hashes.

6 Results

6.1 End-to-End Speed and Success

On AntMaze-Large, HADuS++ reduces wall-clock per episode from 1.42 s to 0.48 s (2.9 $\times$, $p < 0.001$) while increasing success from 78.4 % to 82.3 %. FrankaKitchen records a 2.1 $\times$ speed-up; the 3-D Maze achieves 3.3 $\times$. Confidence intervals are below 4 % of the mean.

6.2 Latent Fidelity

Auto-encoders sharing an identical SE(3)-equivariant decoder but different latents show distortion 9 568 for Euclidean-128 versus 1 381 for Hyperbolic-8, an 85 % reduction despite a 16 $\times$ smaller dimension.

6.3 Adaptive Scheduling Behaviour

In the PointPlanningEnv ADS allocates an average of 10 steps to easy segments and 40 to hard ones, quadrupling effort where needed yet saving 40 % total steps compared with a fixed 20-step schedule. DeepCache produces only a 1.8 $\times$ gap.

6.4 Ablation Study

- **HSD only**: halves the overall speed-up and removes success gains.
- **ADS only**: yields 40 % step reduction but little wall-clock gain due to the 128-d latent cost.
- **No consistency head**: increases failure rate by 3.1 pp in the 3-D Maze.

6.5 Limitations

Success gains are modest; the value critic used by ADS is unavailable in some baselines; GPU memory and `nvprof` FLOP counts were collected for AntMaze but not for Kitchen; robustness to domain randomisation beyond the tested perturbations remains to be studied.

7 Conclusion

We presented HADuS++, a training-free upgrade that combines geometry-aware latent compression with segment-level adaptive scheduling for Hierarchical Diffusion planners. By embedding sub-goals in an 8-d hyperbolic$\otimes$spherical manifold and reallocating denoising effort via an uncertainty-aware rule, HADuS++ cuts high-level FLOPs by up to 50 $\times$ and total wall-clock by 2–3 $\times$ while modestly improving success on three challenging control suites. Ablations confirm that hyperbolic compression reduces latent distortion nearly sevenfold and that ADS shifts four times more compute toward difficult manoeuvres. Future work will (i) learn curvature jointly with downstream rewards, (ii) propagate adaptive scheduling across multiple hierarchy levels, and (iii) integrate model-schedule optimisation techniques such as OMS-DPM Liu et al. [2023] for further acceleration. We believe that aligning latent geometry with task structure and allocating compute where uncertainty is greatest is a promising blueprint for resource-efficient diffusion-based decision makers on embedded platforms and beyond.

References

Jiahe Bai, Baojian Zhou, Deqing Yang, and Yanghua Xiao. Faster local solvers for graph diffusion equations. 2024.

Chang Chen, Fei Deng, Kenji Kawaguchi, Caglar Gulcehre, and Sungjin Ahn. Simple hierarchical planning with diffusion. 2024.

Zheng Chen, Yulun Zhang, Ding Liu, Bin Xia, Jinjin Gu, Linghe Kong, and Xin Yuan. Hierarchical integration diffusion model for realistic image deblurring. 2023.

Xunpeng Huang, Difan Zou, Hanze Dong, Yi Zhang, Yi-An Ma, and Tong Zhang. Reverse transition kernel: A flexible framework to accelerate diffusion inference. 2024.

Senmao Li, Taihang Hu, Joost van de Weijer, Fahad Shahbaz Khan, Tao Liu, Linxuan Li, Shiqi Yang, Yaxing Wang, Ming-Ming Cheng, and Jian Yang. Faster diffusion: Rethinking the role of the encoder for diffusion model inference. 2023.

Ya-Wei Eileen Lin, Ronald R. Coifman, Gal Mishne, and Ronen Talmon. Hyperbolic diffusion embedding and distance for hierarchical representation learning. 2023.

Enshu Liu, Xuefei Ning, Zinan Lin, Huazhong Yang, and Yu Wang. Oms-dpm: Optimizing the model schedule for diffusion probabilistic models. 2023.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free. 2023.

Xinyin Ma, Gongfan Fang, Michael Bi Mi, and Xinchao Wang. Learning-to-cache: Accelerating diffusion transformer via layer caching. 2024.

Taehong Moon, Moonseok Choi, EungGu Yun, Jongmin Yoon, Gayoung Lee, Jaewoong Cho, and Juho Lee. A simple early exiting framework for accelerated sampling in diffusion models. 2024.

Bo Qiang, Yuxuan Song, Minkai Xu, Jingjing Gong, Bowen Gao, Hao Zhou, Weiying Ma, and Yanyan Lan. Coarse-to-fine: a hierarchical diffusion model for molecule generation in 3d. 2023.

Andrew Slavin Ross and Finale Doshi-Velez. Benchmarks, algorithms, and metrics for hierarchical disentanglement. 2021.

Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogeboom. Multistep distillation of diffusion models via moment matching. 2024.

Yanwu Xu, Mingming Gong, Shaoan Xie, Wei Wei, Matthias Grundmann, Kayhan Batmanghelich, and Tingbo Hou. Semi-implicit denoising diffusion models (siddms). 2023.

Hancheng Ye, Jiakang Yuan, Renqiu Xia, Xiangchao Yan, Tao Chen, Junchi Yan, Botian Shi, and Bo Zhang. Training-free adaptive diffusion with bounded difference approximation strategy. 2024.

Zhenyu Zhou, Defang Chen, Can Wang, and Chun Chen. Fast ode-based sampling for diffusion models in around 5 steps. 2023.

[width=0.7] images/wall$_c lock.pdf$

Figure 1: End-to-end wall-clock across algorithms

[width=0.7] images/latent$_d istortion.pdf$

Figure 2: Latent-space distortion (lower is better)

[width=0.7] images/allocated$_s teps.pdf$

Figure 3: Denoising steps versus segment difficulty κ_{seg}